

# Введение в автоматическую обработку текста

Мария Шеянова  
Старший специалист по  
машинному обучению



# Мария Шеянова

О спикере:

- Бакалавриат ФикЛ ВШЭ
- Магистратура по компьютерной лингвистике ВШЭ
- Работаю в Million Agents
- Преподавала NLP в ВШЭ и на летней школе МФТИ



[github.com/maryszmary](https://github.com/maryszmary)



[@maryszmary](https://t.me/maryszmary)



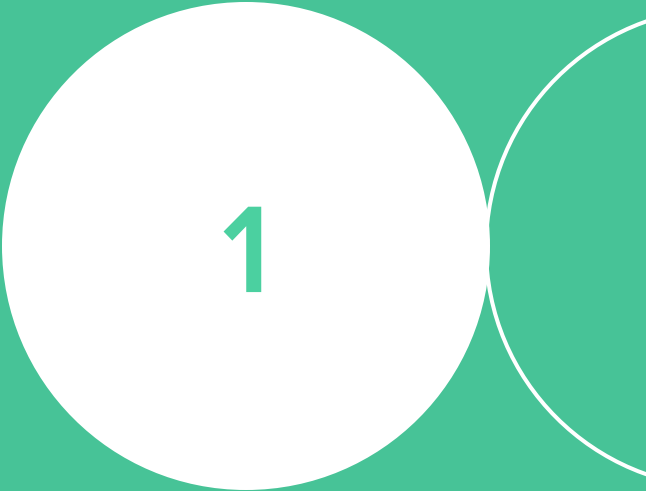
[t.me/maryszmary](https://t.me/maryszmary)



# План занятия

- 1 Основные задачи NLP
- 2 Краткая история становления NLP
- 3 Уровни языка и их обработка
- 4 Законы Ципфа и Хипса
- 5 Векторное представление текста
- 6 Предобработка текста

# Основные задачи NLP



1

# Классификация текстов

В качестве признаков используются в основном слова, тем или иным способом представленные в цифровом виде, то есть в виде векторов

- 1 Фильтрация спама
- 2 Анализ тональности (“настроения” текста)
- 3 Определение интента (цели обращения) в чат-ботах
- 4 Классификация по теме или жанру
- 5 ... что угодно, что можно формализовать как задачу классификации

# Sequence-to-sequence

Модель принимает текст на вход и должна выдать текст. Примеры:

- 1 машинный перевод
- 2 вопросно-ответные системы (чат-боты)
- 3 суммаризация текстов (получить краткий пересказ)
- 4 коррекция написания (спелл-корректоры)

Задача решается, как правило, языковыми моделями.

Почти что угодно можно формализовать как seq-to-seq задачу

# Что ещё можно делать с текстами

## ① Кластеризация текстов, например для:

- агрегации новостей
- выявления закономерностей

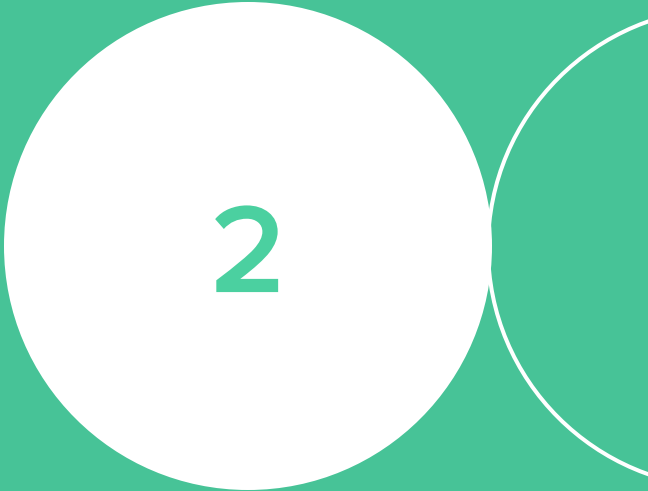
## ② Поиск среди множества документов

## ③ Рекомендации

## ④ Извлечение информации, например:

- NER (извлечение сущностей)
- извлечение фактов и событий
- извлечение ключевых слов

# Краткая история становления NLP



2

# Очень краткий пересказ

- Тест Тьюринга

## Правила (до 90х):

- первые правилковые чатботы
- “мечта” о машинном переводе

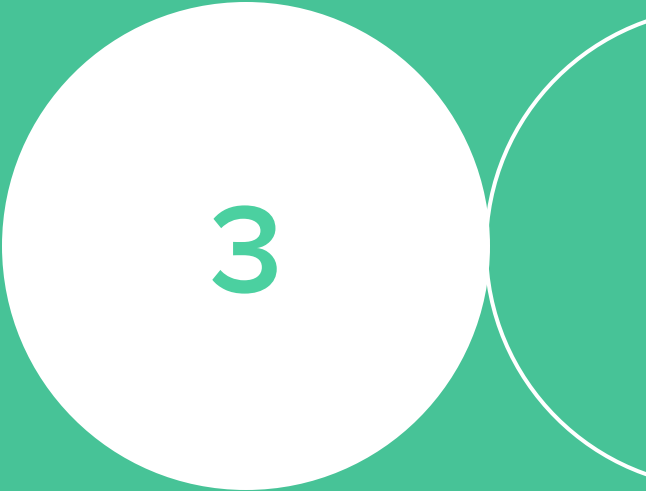
## Счётные модели, простая статистика (до 21 века):

- марковские модели (марковские цепи, HMM)
- TF-IDF для поиска

## Взлёт глубокого обучения (10-е годы 21 века):

- практически все задачи решаются DL, причём хорошо и быстро
- многие классические методы отмирают
- появляются огромные модели

# Уровни языка и их обработка



3

# Низкие уровни

## ① Уровень символов:

- можно работать напрямую с символами (модели fasttext, ELMO)
- токенизация: разбиение текста на слова
- разбиение текста на предложения

## ② Уровень слов и их частей (морфология и лексика):

- использование слов как признаков
- определение частей речи (POS-tagging)
- снятие морфологической неоднозначности

# Высокие уровни

## ① Уровень предложений (синтаксис):

- Выделение именных или глагольных групп (chunking)
- Выделение семантических ролей
- Деревья составляющих и зависимостей

## ② Уровень смысла (семантика и дискурс):

- Разрешение кореферентных связей
- Выделение синонимов
- Анализ аргументативных связей

# Основные проблемы обработки

## ① Неоднозначность

- Лексическая неоднозначность: орган, парить, рожки, атлас
- Морфологическая неоднозначность: Хранение денег в банке. Что делают белки в клетке?
- Синтаксическая неоднозначность: Его удивил простой солдат

## ② Неологизмы: печеньки, заинстаграмить, репостнуть, расшарить, затащить, килорубли

## ③ Разные варианты написания: Россия, Российская Федерация, РФ

## ④ Нестандартное написание: каг дила?

# Законы Ципфа и Хипса

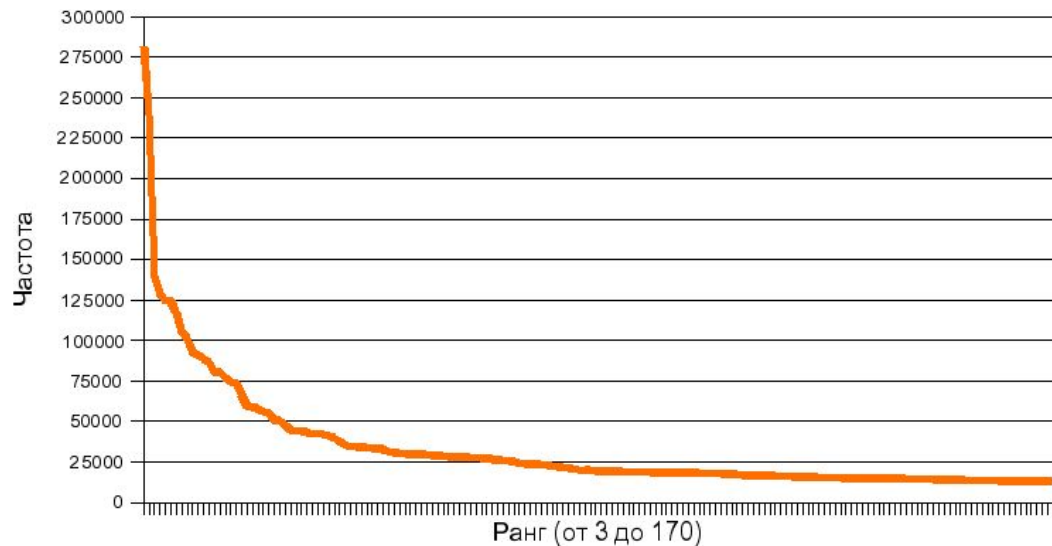


4

# Закон Ципфа

Закономерность: если все слова корпуса текста упорядочить по убыванию частоты их использования, то частота  $n$ -го слова обратно пропорциональна его порядковому номеру  $n$ .

Иными словами, частотность слов убывает очень быстро



# Закон Хипса

Закон Хипса - обратная сторона закона Ципфа:

Чем больше корпус, тем меньше новых слов добавляется с добавлением новых текстов. В какой-то момент **корпус насыщается**

Математически:

С увеличением длины текста (количества токенов), количество слов увеличивается в соответствии с законом:  $|V| = K * N^b$

$N$  – число токенов

$|V|$  – количество слов в словаре

$K, b$  – параметры, обычно  $K \in [10, 100]$ ,  $b \in [0.4, 0.6]$

# Векторное представление текста



5

# One-hot encoding

Упорядочиваем значения категориальной переменной и превращаем её в столько бинарных переменных, сколько у неё было значений

|           |
|-----------|
| garbage   |
| <b>in</b> |
| garbage   |
| out       |



|          |          |          |
|----------|----------|----------|
| 1        | 0        | 0        |
| <b>0</b> | <b>1</b> | <b>0</b> |
| 1        | 0        | 0        |
| 0        | 0        | 1        |

# Мешок слов

А теперь сделаем такой вектор для каждого слова в тексте и сложим!

мягкий  $\rightarrow (1, 0)$

котик  $\rightarrow (0, 1)$

мягкий, мягкий котик  $\rightarrow (2, 1)$

Такой подход называется моделью  
“**мешок слов**”.

Почему? Нам не важен порядок!



# Мешок n-gram

Вместо отдельных слов можно использовать пары, тройки и т.д. подряд идущих слов:

На улице хорошая погода >>

['На улице', 'улице хорошая ', 'хорошая погода']



# TF-IDF векторизация

Вместо количества вхождений слова можно использовать **TF-IDF**.

TF-IDF — способ понять, насколько слово специфично для документа или насколько большую роль слово играет в характеристике документа

Пример:

В тексте научной статьи слово-описание встретилось 7 раз, а слово-кварк — всего 3 раза.

При этом первое слово мало сказало про текст, а после второго стало понятно, о чём он.

Почему так? Слово-описание можно ожидать в какой угодно статье, а кварк — только в ограниченном наборе статей

# TF-IDF векторизация

**term frequency (TF)** — сколько раз слово встретилось в документе

**document frequency (DF)** — в скольких документах встретилось слово

**inverse document frequency (IDF)** — величина, обратно пропорциональная DF

Итог: награждаем слова за количество вхождений в текст и штрафует за количество документов, в которых оно встретилось

## TF-IDF

TF-IDF is a measure of originality of a word by comparing the number of times a word appears in a doc with the number of docs the word appears in.

$$\text{TF-IDF} = \text{TF}(t, d) \times \text{IDF}(t)$$

Term frequency

Inverse document frequency

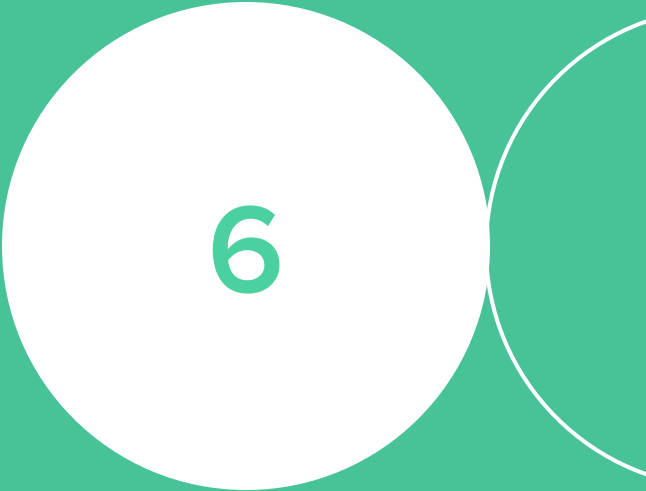
Number of times term  $t$  appears in a doc,  $d$

$$\log \frac{1 + n}{1 + \text{df}(d, t)}$$

# of documents

Document frequency of the term  $t$

# Предобработка текста

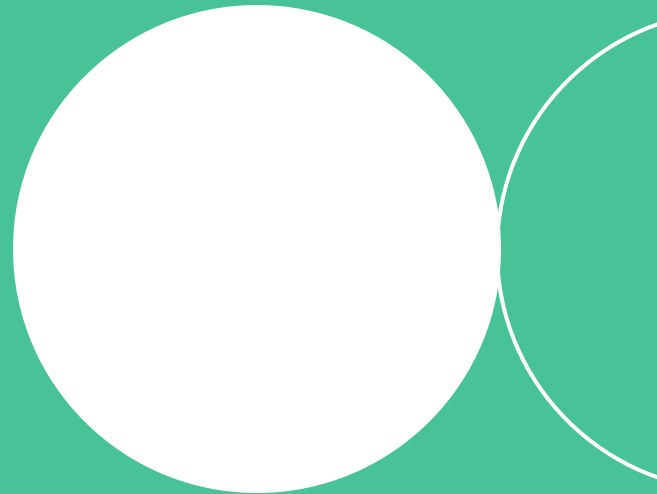


6

# Шаги предобработки

- ① “Чистка” текста:
  - убираем html тэги и прочий мусор
  - приводим к одному регистру
  - иногда — спеллчекинг и прочая работа с написанием
- ② Токенизация (разбиение текста на слова)
- ③ Выкидывание стоп-слов — предлогов, союзов и т.д.
- ④ Иногда — лемматизация

# Итоги



# Итоги

1

## **Множество задач**

Новые задачи продолжают появляться с развитием возможностей

2

## **Относительно молодая сфера**

Последняя революция в области началась около 4 лет назад

3

## **Язык — многоуровневая структура**

4

## **Закон Хипса**

Парето распределение при измерении количественных метрик — частый случай

5

## **Векторизация текста — важная задача**

Только так можно применять методы МО

6

## **Предобработка важна**

# Введение в автоматическую обработку текста



[github.com/maryszmary/](https://github.com/maryszmary/)



[@maryszmary](https://t.me/maryszmary)



[t.me/maryszmary](https://t.me/maryszmary)

Мария Шеянова  
Старший специалист по  
машинному обучению

